# Hana Matsuri: Festival After Dusk — Assignment 3: MVP Implementation

CS549 Artificial Intelligence for Visual Computing · Shweatha Bathina Mallikarjuna (sb2808) · Chiranjeevi Bhuvaneshwari Pamarthy (cp1270) · Jayanth Jayaram Jayakumar Naidu (jj1112) · Prithvi Govindareddy (pg708) · Mentor: Danrui Li

| | |
|---|---|
| **Playable build** | https://hana-matsuri.vercel.app |
| **Demo video** | (YouTube link — to be added) |
| **Source code** | (GitHub link — to be added) (README.md has build instructions) |

## 1. Plan vs. Reality

Our Assignment 2 MVP was a fully playable **Goldfish Scooping (Kingyo-sukui)** activity inside the festival world, with independently moving fish, a responsive paper scoop, fish–scoop collision, real-time water feedback without a full fluid solver, and a result that returns to the festival HUD. The success test was one continuous session: walk to stall → press E → control scoop → water reacts → catch fish → reward and objective complete. This sequence works in the live build.

### Made it in

- **Animated fish:** a rigged goldfish glTF model (8 animation clips) is cloned per fish with its own `AnimationMixer`; slow and fast swim cycles are blended by speed, and the *jump* clip plays when a fish is scooped into the bowl.
- **Fish behaviour:** boids (separation, alignment, cohesion), tub-wall avoidance, and fleeing from the submerged scoop. Fish drift between surface and bottom and dive when frightened, so they stay inside the tank and move independently.
- **Water:** a height-field simulation (discrete 2D wave equation on a 96×41 grid, fixed 60 Hz step, damping, reflecting borders). Dipping, dragging and lifting the scoop, caught fish and shallow-swimming fish create ripples that spread and reflect off the tub walls; vertex normals and crest colouring are rebuilt each step.
- **Collision:** a fish is caught only if its body circle overlaps the paper disc *and* it is above the paper's depth when the scoop is lifted. Paper strength drops with water drag (scoop speed) and fish weight, and the paper visibly tears. A debug view (V) draws the catch radius and every fish's hit circle (green = catchable, red = too deep).
- **Game loop:** results return to the festival: yen reward, prize, objective ticked, and a firework pattern unlocked for the finale.
- **Performance tools:** live FPS, fish count, grid size and simulation cost in the HUD; P adds 20 fish, O raises the water resolution.

### Cut or changed

- We planned "ripple or height-field water"; we implemented the height field and dropped true refraction of the fish under the surface. The water is a translucent, reflective surface instead.
- Fish collision uses 2D circles plus a depth check rather than mesh-accurate tests. It is cheaper and easier to read on screen.
- The goldfish model's glass-like transmission material was disabled: it forced an extra full-scene render pass every frame.
- Large assets were compressed for the web: the festival street from 97 MB to 23 MB and the goldfish from 12 MB to 3.5 MB.

**Performance (Chrome, desktop)**

| Scenario | Fish | Water grid | Water sim cost | FPS |
|---|---|---|---|---|
| Normal play | 16 | 96 × 41 | ≈1.3 ms/frame | 60 |
| Stress test | 216 | 384 × 164 | ≈14 ms/frame | ≈31 |

Normal play meets our 60 FPS target. Under stress it stays at our 30 FPS minimum; the main costs are skinned-mesh animation (one mixer per fish) and the CPU wave simulation, which grows with the number of grid cells.

## 2. AI Utility

- **Claude Code** was the most useful tool. It produced working first versions of large systems quickly: the boids steering, the height-field water, the behaviour-tree NPCs, A* navigation, the fireworks particles, and the asset compression pipeline (gltf-transform). It also ran the game in a headless browser to take screenshots and catch runtime errors.
- **ChatGPT** helped with debugging questions, Three.js API details and shader/particle ideas.
- **AI image generation** was used only for concept mockups (visual reference), not for in-game assets.
- **Time saved vs. debugging:** overall AI saved a lot of time on boilerplate and first drafts. The cost was in integration and tuning. Examples: the first night lighting was badly over-exposed and needed manual tuning; the default model compression merged meshes and broke our collision and lantern-glow lookups, so we had to change its settings; the goldfish model's material settings had to be found and disabled for performance; and game-feel values (scoop strength, flee distance, ripple size) still needed playtesting. AI output always needed checking in the running game.
- No runtime AI API is used, as stated in our tech spec, so the build is predictable and works offline.

## 3. Roadmap to Final

- **Other main activities:** polish the Shooting Gallery (cork ballistics, toppling prizes) and the Taiko rhythm game, and finish Kimodameshi (scares, courage seals, fog), which are currently playable prototypes.
- **Rendering:** enable the island environment with reflective water, cliffs and the raised shrine terrace (drafted in `island.js`); add caustics or refraction to the goldfish tub; tune night lighting.
- **NPCs:** extend the behaviour trees (queueing at stalls, reacting to the player's games) and the stall-owner states.
- **Short interactions:** ring toss, yo-yo balloon fishing, mask memory, Daruma balancing, lantern-lighting puzzle (scope permitting).
- **Finale:** make firework patterns depend on more of the player's progress; performance pass (instanced fish, GPU water) for lower-end laptops.